Landing Motion Foundation

Landing Motion Foundation — nền GSAP cho landing & block

> Một nền animation DUY NHẤT cho mọi landing page. Mọi block tuân theo cùng contract → animation đồng bộ, chạy giống nhau ở **editor preview** lẫn trang public `/p/[slug]`, kể cả hiệu ứng scroll đặc biệt (pin / scrub / parallax).
>
> File: `src/components/modules/block-editor/motion/LandingMotion.tsx` Áp dụng tham chiếu: 5 block `mint_*` (`blocks/Mint*.tsx`).

---

1. Vì sao có nền này

Trước đây mỗi block tự setup GSAP/ScrollTrigger với giả định riêng:

- tự `gsap.registerPlugin` + tự `gsap.context` ;
- `window.matchMedia` để biết mobile (sai trong khung editor 390/834/1280);
- mặc định scroll theo `window` (sai khi block nằm trong canvas editor cuộn nội bộ);
- không ai gọi `ScrollTrigger.refresh()` sau khi ảnh load → scrub/pin **đo sai vị trí** → **"không chạy"**.

Hệ quả: animation flaky, editor ≠ `/p` . Nền này gom tất cả phần chung về một chỗ và buộc block dùng chung qua hook.

---

2. Hai mảnh

`LandingMotionRoot` — provider cấp trang (đặt 1 lần)

Sở hữu toàn bộ phần dùng chung:

- `gsap.registerPlugin(ScrollTrigger)` (1 lần).
- **Lenis** smooth-scroll (desktop, khi scroller = window) + `lenis.on('scroll', ScrollTrigger.update)` + đồng bộ `gsap.ticker` .
- **Refresh tập trung**: `ScrollTrigger.refresh()` khi mount / `window.load` / `document.fonts.ready` / `ResizeObserver` đổi kích thước (bắt layout-shift do ảnh load). Đây là thứ trước đây thiếu.
- Cấp đúng **`scroller`** theo ngữ cảnh: `null` = `window` (cho `/p`) hoặc phần tử canvas (cho editor).

Đã wire sẵn 2 nơi:

Nơi	File	scroller	Lenis
Public `/p/[slug]`	`app/p/[id]/_block-render.tsx`	`window`	bật (desktop)

Nơi	File	scroller	Lenis
Editor canvas	`block-editor/BlockPreview.tsx`	phần tử surface (`scrollerRef`)	tắt (native)

```
// /p
<LandingMotionRoot enableSmooth={isVerticalTransition(transition)}>
  <main className='w-full'>{items}</main>
</LandingMotionRoot>

// editor
<LandingMotionRoot scrollerRef={surfaceRef} enableSmooth={false}>
  {/* canvas + blocks */}
</LandingMotionRoot>
```

`useBlockMotion` — contract cho block

Thay cho `useEffect + gsap.context` thủ công. Hook tự: tạo `gsap.context` scope theo root block, tiêm `scroller`, cấp `isMobile` (đo theo container, không viewport), `revert()` khi unmount, gọi `requestRefresh()` sau build.

```
const rootRef = React.useRef<HTMLElement>(null);

useBlockMotion(rootRef, ({ scroller, isMobile, requestRefresh }) => {
  if (isMobile) return;                  // nhánh desktop-only nếu cần
  const el = rootRef.current!.querySelector('.js-thing');
  gsap.to(el, {
    y: -60,
    scrollTrigger: {
      trigger: el,
      scroller,                          // BẮT BUỘC — đồng bộ /p & editor
      start: 'top bottom',
      end: 'bottom top',
      scrub: 1,
      invalidateOnRefresh: true,
    },
  });
}, [/* deps tùy chọn, vd [variant] */]);
```

Block **phức tạp** (tự quản context + cleanup riêng, vd hero canvas frame-scrub): dùng `useLandingMotion()` lấy `scroller` + `requestRefresh`, vẫn tuân nền:

```
const { scroller, requestRefresh } = useLandingMotion();
React.useEffect(() => {
  // ... load ảnh / canvas ...
  img.onload = () => { if (done) requestRefresh(); };   // refresh sau media load
  const ctx = gsap.context(() => {
    gsap.to(state, { scrollTrigger: { trigger: '.intro', scroller, scrub: 1, pin: '.canvas' } });
  }, rootRef);
  return () => ctx.revert();
}, [scroller, requestRefresh /* , isMobile ... */]);
```

---

3. RULE cho block (contract — bắt buộc)

#	Rule
R1	Mọi scroll-animation qua `useBlockMotion` (hoặc `useLandingMotion` cho block phức tạp). **CẤM** tự `new Lenis`, tự nghe `window` scroll, hay `gsap.registerPlugin` trong block.
R2	Responsive bằng **container query** (`@2xl:` , `@4xl:` , đơn vị `cqi`) + `isMobile` theo container. **KHÔNG** dùng viewport `md:` / `lg:` hay `window.matchMedia` .
R3	Mọi `scrollTrigger` PHẢI truyền `scroller` (từ api). Thêm `invalidateOnRefresh: true` .
R4	Sau khi media (ảnh/canvas) load xong → gọi `requestRefresh()` (block đơn giản: hook tự lo; block phức tạp: gọi tay trong `onload`).
R5	Overlay/drawer **không** đặt `position: fixed` tùy tiện; **không** đặt `container-type` lên tổ tiên của phần `pin` / `fixed` (làm vỡ pin + neo sai).
R6	Idempotent + cleanup: block đơn giản hook tự `revert` ; block phức tạp tự `return () => ctx.revert()` + gỡ listener/observer.

4. Thêm 1 block mới (checklist)

1. `types.ts` : thêm `BlockType` + `interface XxxContent` + vào union `BlockContent` + `BLOCK_TYPE_META` .
2. `factory.ts` : default content (VI/EN).
3. `blocks/Xxx.tsx` : renderer — `@container` root + `data-element-key` cho element + `useBlockMotion` cho animation (R1–R6).
4. `blocks/renderBlock.tsx` : `case 'xxx'` → `<Xxx content=... styles=... activeKey=.../>` .
5. `AddBlockDialog.tsx` : thêm vào `ORDERED_TYPES` .
6. `LayersPanel.tsx` : `ELEMENTS_BY_TYPE['xxx']` (khai báo lớp để chọn/sửa inline). Element mảng (vd list) → thêm kind + control trong `ElementToolbar.tsx` .
7. **BE**: seed 1 row `blocks` (scope `system`) cho `type_code` mới — nếu không, commit version sẽ 400 `unknown block type` (xem migration `000038` +).

5. Caveat đã biết

- **Editor scrub chỉ chuẩn ở zoom 100%**: canvas dùng `transform: scale()` theo zoom → ScrollTrigger đo bị nhân scale ở zoom ≠ 100%. Authoring chấp nhận được; output thật `/p` luôn đúng.
- **Animation scroll trong editor**: editor hiện 1 block/lần, không có flow cuộn cả trang → pin/scrub xuyên-section chỉ thấy đầy đủ trên `/p` .
- `@container` **trên block root** tạo containing-block cho `position:fixed` (drawer) → cần portal/đưa overlay ra ngoài nếu muốn full-viewport (theo R5).

6. Tham chiếu

- Nền: `src/components/modules/block-editor/motion/LandingMotion.tsx`
- Hook container-mobile: `src/components/modules/block-editor/blocks/useContainerMobile.ts`

- Block mẫu (đơn giản): `blocks/MintDetails.tsx`, `MintSku.tsx`, `MintVariants.tsx`, `MintDiscover.tsx`
- Block mẫu (phức tạp, canvas): `blocks/MintHero.tsx`
- Wire `/p`: `src/app/p/[id]/_block-render.tsx`
- Wire editor: `src/components/modules/block-editor/BlockPreview.tsx`